

■ PANDAS

Complete Revision Guide

From Series fundamentals to DataFrames, CSV/JSON I/O, indexing, filtering, aggregation, groupBy, and data cleaning — everything from your notebook, extended with key concepts you need to know.

Contents

- 1 Setup & pandas Basics
- 2 Series
- 3 DataFrame
- 4 File I/O (CSV · JSON)
- 5 Selecting & Indexing
- 6 Filtering Data
- 7 Aggregate Functions
- 8 GroupBy
- 9 Data Cleaning
- 10 Quick-Reference Cheat Sheet

1 · Setup & Pandas Basics

Pandas is a fast, flexible, and expressive data-analysis library built on top of NumPy. It introduces two core data structures: **Series** (1-D labelled array) and **DataFrame** (2-D labelled table). Almost every data-science project starts with pandas.

Importing & version check

```
import pandas as pd

print(pd.__version__) # e.g. 2.2.2

2.2.2
```

■ NOTE

Always import pandas as **pd** — this is universal convention. Pandas 2.x introduced Copy-on-Write semantics, making chained operations safer.

2 · Series

A Series is a **one-dimensional array with labels (index)**. Think of it as a single column of a spreadsheet. Every element has a positional integer index AND a custom label.

2.1 Creating a Series

From a Python list with custom labels:

```
data = [343, 344, 3454.4, 564, 343]
series = pd.Series(data, index=['a', 'barbie', 'c', 'd', 'e'])
print(series)

a          343.0
barbie     344.0
c          3454.4
d           564.0
e           343.0
dtype: float64
```

From a Python dictionary (keys become the index automatically):

```
diet = {'day1': 420, 'day2': 380, 'day3': 390}
series = pd.Series(diet)
print(series)

day1      420
day2      380
day3      390
dtype: int64
```

■ NOTE

When you mix int and float values in a list, pandas auto-promotes the dtype to float64. Dict-based Series preserves insertion order (Python 3.7+).

2.2 Accessing elements

```
# .loc — label-based access
print(series.loc['c'])    # 3454.4

# .iloc — position-based access (0-indexed)
print(series.iloc[3])     # 564.0
```

2.3 Updating values

```
series.loc['c'] = 111.02
print(series.loc['c'])    # 111.02
```

2.4 Boolean / conditional filtering

```
# Keep only values > 554, then add 434
print(series[series > 554] + 434)
```

```
d      998.0
dtype: float64
```

This is called **Boolean indexing**: the condition `series > 554` creates a boolean mask; applying it selects only the matching rows. You can chain arithmetic directly.

2.5 Key Series attributes & methods

Method / Attribute	Description
<code>series.dtype</code>	Data type of elements
<code>series.shape</code>	Tuple (n,) — number of elements
<code>series.index</code>	The index labels
<code>series.values</code>	NumPy array of values
<code>series.head(n)</code>	First n rows (default 5)
<code>series.tail(n)</code>	Last n rows
<code>series.describe()</code>	Count, mean, std, min, 25%, 50%, 75%, max
<code>series.unique()</code>	Array of unique values
<code>series.value_counts()</code>	Frequency of each unique value
<code>series.isnull()</code>	Boolean mask — True where NaN
<code>series.dropna()</code>	Remove NaN values
<code>series.fillna(val)</code>	Replace NaN with val

3 · DataFrame

A DataFrame is a **2-D table** — think of it as multiple Series sharing the same index. Each column is a Series. A DataFrame has both a **row index** and **column labels**.

3.1 Creating a DataFrame

```
character = {
    'Name': ['Rudra', 'Pawan', 'Rashmi'],
    'Age' : [39, 44, 23]
}
df = pd.DataFrame(character, index=['Stud 1', 'Stud 2', 'Stud 3'])
print(df)
```

	Name	Age
Stud 1	Rudra	39
Stud 2	Pawan	44
Stud 3	Rashmi	23

3.2 Adding a new column

```
df['Job'] = ['Engineer', 'Doctor', 'Teacher']
print(df)
```

	Name	Age	Job
Stud 1	Rudra	39	Engineer
Stud 2	Pawan	44	Doctor
Stud 3	Rashmi	23	Teacher

■ GOTCHA

Assigning a new column name **creates** it if it doesn't exist, or **overwrites** it if it does. The list length must match the number of rows.

3.3 Adding new rows with `pd.concat()`

```
new_row = pd.DataFrame(
    [{'Name': 'Sandeep', 'Age': 34, 'Job': 'Engineer'}],
    index=['Stud 4']
)
df = pd.concat([df, new_row]) # append a single row

# Append multiple rows at once
more_rows = pd.DataFrame(
    [{'Name': 'Priya', 'Age': 28, 'Job': 'Designer'},
    {'Name': 'Arjun', 'Age': 31, 'Job': 'DevOps'}],
    index=['Stud 5', 'Stud 6']
)
df = pd.concat([df, more_rows])
```

■ GOTCHA

The old `df.append()` method was removed in Pandas 2.0. Always use `pd.concat()` instead. Pass `ignore_index=True` if you want fresh integer indices after concatenation.

3.4 Saving to CSV

```
df.to_csv('students.csv', index=True)  # index=False to omit row labels
```

3.5 Useful DataFrame attributes

Attribute / Method	Returns
df.shape	(rows, cols) tuple
df.dtypes	Data type of every column
df.columns	Column label Index object
df.index	Row label Index object
df.head(n)	First n rows
df.tail(n)	Last n rows
df.info()	Summary: dtypes, nulls, memory
df.describe()	Statistics for numeric columns
df.to_string()	Full string repr (no truncation)
df.copy()	Deep copy of the DataFrame

4 · File I/O (CSV · JSON · Excel)

Pandas can read from and write to dozens of formats. The three most common are CSV, JSON, and Excel. The Pokemon dataset used throughout your notebook comes from a CSV file.

4.1 CSV

```
# Read
df = pd.read_csv('new.csv')          # default – integer index
df = pd.read_csv('new.csv', index_col='Name') # use Name column as index

# Write
df.to_csv('output.csv', index=True)    # include index
df.to_csv('output.csv', index=False)  # omit index column
```

■ TIP

Using `index_col='Name'` lets you do `df.loc['Pikachu']` directly instead of filtering by value. Always check whether your CSV already has an index column to avoid a duplicate 'Unnamed: 0' column.

4.2 JSON

```
# Read
df = pd.read_json('output.json')

# Write
df.to_json('output.json', orient='records', indent=2)
```

4.3 Excel (bonus)

```
# Read (requires openpyxl)
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')

# Write
df.to_excel('output.xlsx', sheet_name='Pokemon', index=False)
```

■ NOTE

Install openpyxl separately: `pip install openpyxl`

4.4 Useful read parameters

Parameter	Effect
<code>sep='\t'</code>	Tab-separated (TSV)
<code>header=None</code>	No header row – pandas creates 0,1,2...
<code>names=[...]</code>	Override column names
<code>usecols=[...]</code>	Load only specific columns
<code>nrows=100</code>	Read only first 100 rows (fast preview)
<code>na_values=['N/A','?']</code>	Treat these strings as NaN
<code>dtype={'Age': int}</code>	Force dtype on load
<code>parse_dates=['date']</code>	Parse column as datetime

5 · Selecting & Indexing

Pandas has two primary indexers: `.loc` (label-based) and `.iloc` (integer position-based). Understanding the difference is critical — they behave differently for slicing and for DataFrames that use a non-integer index.

5.1 Selecting columns

```
# Single column → returns a Series
df['Name']

# Multiple columns → returns a DataFrame
df[["Name", "Height", "Weight"]]

# Suppress truncation for large output
print(df["Name"].to_string())
```

5.2 .loc — label-based selection

```
# Single row by label
print(df.loc["Pikachu"])

# Multiple columns of one row
print(df.loc["Snorlax", ["Height", "Weight"]])

# Row slice (INCLUSIVE on both ends with .loc)
print(df.loc["Snorlax":"Dratini", ["Height", "Weight"]])
```

```
# Output for Snorlax:Dratini slice
      Height  Weight
Name
Snorlax    2.1   460.0
Articuno    1.7    55.4
Zapdos      1.6    52.6
Moltres     2.0    60.0
Dratini     1.8     3.3
```

■ GOTCHA

`.loc` slicing is **inclusive on both ends**. So `df.loc["Snorlax":"Dratini"]` includes Dratini in the result.

5.3 .iloc — position-based selection

```
# Row 2 by position (0-indexed)
print(df.iloc[2])

# Slice: every 2nd row from 0..9, columns 0..2
print(df.iloc[:10:2, 0:3])
```

```
# output for iloc[:10:2, 0:3]
      No  Type1  Type2
Name
Bulbasaur    1  Grass  Poison
Venusaur     3  Grass  Poison
Charmeleon    5   Fire    NaN
Squirtle     7  Water    NaN
Blastoise    9  Water    NaN
```

■ NOTE

`.iloc` follows Python slicing rules: **exclusive end**. `df.iloc[0:3]` returns rows 0, 1, 2 — not row 3.

5.4 Safe lookup with error handling

```
pokemon = input("Enter a pokemon name: ")
try:
    print(df.loc[pokemon])
except KeyError:
    print("Pokemon not found")
```

Good practice when the index may not contain the queried label — avoids crashing your program.

5.5 `.loc` vs `.iloc` — Summary

Feature	<code>.loc</code> (label)	<code>.iloc</code> (position)
Reference by	Index labels/names	Integer position 0,1,2...
Slice end	Inclusive	Exclusive (Python-style)
Accepts boolean array	Yes	Yes
Works on integer index	label == position!	Always positional

6 · Filtering Data (Boolean Indexing)

Boolean indexing is one of the most powerful pandas patterns. You pass a condition that returns a boolean Series, and pandas keeps only the True rows.

6.1 Single condition

```
# Pokemon with Height >= 2
tall = df[df["Height"] >= 2]

# Heavy Pokemon (Weight >= 90)
heavy = df[df["Weight"] >= 90]

# Legendary only
legendaries = df[df["Legendary"] == 1]
```

6.2 Multiple conditions

```
# OR — use | (not 'or')
grass_types = df[(df["Type1"] == "Grass") | (df["Type2"] == "Grass")]

# AND — use & (not 'and')
grass_water = df[(df["Type1"] == "Grass") & (df["Type2"] == "Water")]

# NOT — use ~
non_legendary = df[~(df["Legendary"] == 1)]
```

■ GOTCHA

Always wrap each condition in **parentheses ()** when combining with | or &. Python's operator precedence will give wrong results otherwise.

6.3 .isin() — match a list of values

```
fire_or_water = df[df["Type1"].isin(["Fire", "Water"])]

# Equivalent to:
# df[(df["Type1"]=="Fire") | (df["Type1"]=="Water")]
```

6.4 String contains (.str accessor)

```
# Find all pokemon whose name contains 'saur' (case-insensitive)
saur = df[df["Name"].str.contains("saur", case=False, na=False)]
```

■ TIP

Use **na=False** so NaN values in the column don't raise an error — they'll just be treated as non-matching.

7 · Aggregate Functions

Aggregate (or reduction) functions collapse a column or entire DataFrame down to a single summary value. They ignore NaN by default.

7.1 Whole DataFrame

```
print(df.mean(numeric_only=True))
print(df.sum(numeric_only=True))
print(df.min(numeric_only=True))
print(df.max(numeric_only=True))
print(df.count())           # non-NaN count per column
```

```
# mean output (sample)
No          75.500000
Height      1.200000
Weight      46.231333
Legendary   0.026667
dtype: float64
```

```
# count output (shows Type2 has only 67 non-null)
Type2      67
...
```

■ NOTE

Pass **numeric_only=True** to mean/sum/min/max to skip string columns, which can't be averaged. In older pandas this was the default; in newer versions you must be explicit.

7.2 Single column

```
print(df["Height"].mean())    # 1.2
print(df["Height"].sum())     # 180.0
print(df["Weight"].min())     # 0.1
print(df["Weight"].max())     # 460.0
print(df["Weight"].count())   # 150
```

7.3 describe() — full statistics at once

```
print(df["Height"].describe())
```

```
count    150.000000
mean      1.200000
std       0.908...
min       0.200000
25%       0.600000
50%       1.000000
75%       1.600000
max       8.800000
Name: Height, dtype: float64
```

7.4 Quick reference table

Function	Description
df.mean()	Arithmetic mean
df.median()	Middle value (50th percentile)
df.std()	Standard deviation

<code>df.var()</code>	Variance
<code>df.sum()</code>	Sum of all values
<code>df.min()</code> / <code>df.max()</code>	Smallest / largest value
<code>df.count()</code>	Non-NaN count
<code>df.nunique()</code>	Count of unique values
<code>df.idxmin/idxmax()</code>	Label of min/max row
<code>df.corr()</code>	Pairwise correlation matrix (numeric cols)
<code>df.cumsum()</code>	Cumulative sum (returns Series)

8 · GroupBy

GroupBy follows the **split** → **apply** → **combine** paradigm. You split the data by a key, apply a function to each group, and combine the results.

8.1 Basic groupby

```
group = df.groupby("Type1")

# Mean height per type
print(group["Height"].mean())

# Min height per type
print(group["Height"].min())

# sample output
Type1
Bug      0.900000
Dragon   2.666667
Electric 0.855556
...
Water    1.300000
Name: Height, dtype: float64
```

8.2 Multiple aggregations (.agg)

```
result = df.groupby("Type1")["Height"].agg(["mean", "min", "max", "count"])
print(result)
```

8.3 Group by multiple columns

```
result = df.groupby(["Type1", "Type2"])["Weight"].mean()
print(result)
```

8.4 Iterating over groups

```
for type_name, group_df in df.groupby("Type1"):
    print(f"Type: {type_name}, Count: {len(group_df)}")
```

■ TIP

GroupBy is lazy — it doesn't compute anything until you apply an aggregation. You can chain multiple operations:

```
df.groupby("Type1")["Weight"].mean().sort_values(ascending=False)
```

9 · Data Cleaning

Real-world data is messy. Pandas provides a rich toolkit for fixing it: dropping columns/rows, handling missing values, replacing values, standardising strings, fixing dtypes, and removing duplicates.

9.1 Dropping columns or rows

```
# Drop one column (non-destructive by default)
new_df = df.drop(columns=["Legendary"])

# Drop multiple columns
new_df = df.drop(columns=["Type2", "No"])

# Drop rows by label
new_df = df.drop(index=["Stud 4", "Stud 5"])

# In-place modification (modifies df itself)
df.drop(columns=["Temp"], inplace=True)
```

9.2 Handling missing values (NaN)

```
# Check for nulls
df.isnull().sum()          # count NaNs per column
df.isnull().any()          # True/False per column

# Drop rows where Type2 is NaN
new_df = df.dropna(subset=["Type2"])

# Drop rows where ANY column is NaN
new_df = df.dropna()

# Drop columns that have ANY NaN
new_df = df.dropna(axis=1)

# Fill NaN with a specific value
new_df = df.fillna({"Type2": "None"})

# Forward fill (propagate last valid value forward)
new_df = df.fillna(method='ffill')

# Fill with column mean
df["Weight"].fillna(df["Weight"].mean(), inplace=True)
```

9.3 Replacing values

```
# Replace specific values in a column
new_df["Type1"] = new_df["Type1"].replace({"Grass": "Herbivorous", "Fire": "Firefly"})

# Replace across entire DataFrame
df.replace({"NaN": None}, inplace=True)
```

9.4 String standardisation (.str accessor)

```
# Lowercase all names
new_df["Name"] = new_df["Name"].str.lower()

# Uppercase
new_df["Name"] = new_df["Name"].str.upper()

# Strip leading/trailing whitespace
new_df["Name"] = new_df["Name"].str.strip()

# Replace substring
new_df["Name"] = new_df["Name"].str.replace("_", " ")
```

9.5 Fixing data types (.astype)

```
# Convert Legendary column from int to bool
new_df["Legendary"] = new_df["Legendary"].astype(bool)

# Convert to integer
df["No"] = df["No"].astype(int)

# Convert to string
df["Type1"] = df["Type1"].astype(str)

# Convert to datetime
df["Date"] = pd.to_datetime(df["Date"])
```

9.6 Removing duplicates

```
new_df = df.drop_duplicates() # drop exact duplicate rows

# Drop rows where 'Name' is duplicated
new_df = df.drop_duplicates(subset=["Name"])

# Keep last occurrence instead of first
new_df = df.drop_duplicates(subset=["Name"], keep="last")

# Check for duplicates
df.duplicated().sum() # number of duplicate rows
```

9.7 Renaming columns

```
df.rename(columns={"Name": "PokemonName", "No": "PokedexNo"}, inplace=True)

# Rename all columns at once (provide list of same length)
df.columns = ["id", "name", "type1", "type2", "height", "weight", "is_legendary"]
```

10 · Quick-Reference Cheat Sheet

■ CREATION

Syntax	Description
<code>pd.Series(list, index=[...])</code>	Series from list
<code>pd.Series(dict)</code>	Series from dict
<code>pd.DataFrame(dict, index=[...])</code>	DataFrame from dict
<code>pd.read_csv('f.csv')</code>	Read CSV
<code>pd.read_json('f.json')</code>	Read JSON
<code>pd.read_excel('f.xlsx')</code>	Read Excel

■ INSPECTION

Syntax	Description
<code>df.shape / df.dtypes</code>	Dimensions / types
<code>df.head(n) / df.tail(n)</code>	First / last n rows
<code>df.info()</code>	Types + null counts
<code>df.describe()</code>	Numeric stats
<code>df.isnull().sum()</code>	NaN count per column

■ SELECTION

Syntax	Description
<code>df['col']</code>	Single column (Series)
<code>df[['c1', 'c2']]</code>	Multiple columns (DataFrame)
<code>df.loc[row_label]</code>	Row by label
<code>df.iloc[row_pos]</code>	Row by position
<code>df.loc['a':'c', 'col']</code>	Slice rows + column
<code>df.iloc[:,2, 0:3]</code>	Every other row, cols 0-2

■ FILTERING

Syntax	Description
<code>df[df['col'] > val]</code>	Condition filter
<code>df[(cond1) & (cond2)]</code>	AND – both true
<code>df[(cond1) (cond2)]</code>	OR – either true
<code>df[~(cond)]</code>	NOT – negate
<code>df[df['col'].isin([v1,v2])]</code>	Match list
<code>df[df['col'].str.contains('pat')]</code>	Substring match

■ AGGREGATION

Syntax	Description
<code>df.mean/sum/min/max/count()</code>	Reduce whole DataFrame

<code>df['col'].mean()</code>	Reduce single column
<code>df['col'].describe()</code>	Full stats
<code>df.groupby('col')['c'].mean()</code>	GroupBy + aggregate
<code>df.groupby('col')['c'].agg([...])</code>	Multiple aggregations

■ CLEANING

Syntax	Description
<code>df.drop(columns=['c'])</code>	Drop column(s)
<code>df.dropna(subset=['c'])</code>	Drop rows with NaN in c
<code>df.fillna({'c': val})</code>	Fill NaN with value
<code>df['c'].astype(dtype)</code>	Change dtype
<code>df['c'].replace({old:new})</code>	Replace values
<code>df['c'].str.lower()</code>	Lowercase strings
<code>df.drop_duplicates()</code>	Remove duplicate rows
<code>df.rename(columns={...})</code>	Rename columns

■ SAVING

Syntax	Description
<code>df.to_csv('f.csv', index=False)</code>	Save as CSV
<code>df.to_json('f.json', orient='records')</code>	Save as JSON
<code>df.to_excel('f.xlsx', index=False)</code>	Save as Excel